

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**BÁO CÁO
BÀI TẬP LỚN TRÍ TUỆ NHÂN TẠO**

**ĐỀ TÀI:
XÂY DỰNG GAME CARO AI SỬ DỤNG MINIMAX VÀ
ALPHA-BETA PRUNING**

**Giảng viên hướng dẫn: Vũ Đình Nam
Nguyễn Phương Thái**

**Thành viên nhóm: Đặng Trung Hiếu – 23020269
Nguyễn Minh Hiếu – 23020272**

LỜI NÓI ĐẦU

Trong môn học Trí tuệ nhân tạo, các bài toán trò chơi đối kháng là một nội dung quan trọng giúp sinh viên hiểu rõ hơn về cách máy tính đưa ra quyết định và lựa chọn nước đi tối ưu. Trong đó, game cờ Caro là một trò chơi quen thuộc, có luật chơi đơn giản nhưng vẫn đòi hỏi khả năng tính toán và xây dựng chiến thuật hợp lý.

Trong bài tập lớn này, nhóm thực hiện xây dựng chương trình game Caro bằng ngôn ngữ Python kết hợp thư viện pygame để tạo giao diện đồ họa. Chương trình áp dụng thuật toán Minimax và Alpha-Beta pruning nhằm xây dựng AI có khả năng chơi với người dùng.

Thông qua quá trình thực hiện đề tài, nhóm có cơ hội tìm hiểu rõ hơn về:

- cây trò chơi (Game Tree)
- thuật toán Minimax
- kỹ thuật cắt tỉa Alpha-Beta
- hàm đánh giá heuristic
- tìm kiếm đối kháng trong trí tuệ nhân tạo

Ngoài ra, đề tài cũng giúp nhóm rèn luyện thêm kỹ năng lập trình, tổ chức chương trình và phân tích thuật toán trong thực tế.

Do thời gian thực hiện và kiến thức còn hạn chế, báo cáo chắc chắn vẫn còn những thiếu sót. Nhóm rất mong nhận được những góp ý từ giảng viên để có thể hoàn thiện đề tài tốt hơn.

Nhóm xin chân thành cảm ơn!

MỤC LỤC

NỘI DUNG	4
Phần 1: Giới thiệu đề tài và yêu cầu bài toán	4
1.1. Giới thiệu bài toán cờ Caro.....	4
1.2. Mục tiêu đề tài.....	4
1.3. Các mức độ và yêu cầu chức năng của chương trình.....	5
Phần 2: Phân tích và thiết kế thuật toán	6
2.1. Luật chơi và điều kiện kết thúc.....	6
2.2. Biểu diễn trạng thái bàn cờ (State Representation).....	6
2.3. Cơ chế sinh nước đi hợp lệ và tối ưu không gian tìm kiếm.....	6
2.4. Hàm kiểm tra trạng thái kết thúc (Terminal Test).....	8
2.5. Hàm đánh giá trạng thái (Heuristic Evaluation Function).....	9
2.6. Cài đặt thuật toán Minimax.....	10
2.7. Cài đặt thuật toán cải tiến Alpha-Beta Pruning.....	12
2.8. Thiết kế giao diện chương trình.....	13
Phần 3: Thực Nghiệm và phân tích đánh giá	15
3.1. Thiết kế các kịch bản điểm thử.....	15
3.2. Bảng kết quả thực nghiệm so sánh.....	19
3.3. Nhận xét và phân tích kết quả thực nghiệm.....	19
3.4. Ưu điểm và hạn chế của chương trình.....	20
Phần 4: Kết luận và hướng phát triển	22
4.1. Kết luận.....	22
4.1. Hướng phát triển nâng cao.....	22
Danh sách tài liệu tham khảo	23

NỘI DUNG

Phần 1: Giới thiệu đề tài và yêu cầu bài toán

1.1. Giới thiệu bài toán cờ Caro

Trong Trí tuệ nhân tạo, cờ Caro là một bài toán kinh điển thuộc nhóm trò chơi đối kháng hai người, thông tin hoàn hảo và có tổng bằng không (Zero-sum game). Do không có yếu tố may rủi, kết quả ván đấu hoàn toàn phụ thuộc vào chiến thuật và khả năng tính toán nước đi của mỗi bên.

Về mặt kỹ thuật, cờ Caro sở hữu không gian trạng thái vô cùng lớn. Ngay cả với kích thước tối thiểu 9×9 , số lượng thế cờ khả thi vẫn tăng trưởng theo cấp số nhân sau mỗi lượt đi, gây ra hiện tượng bùng nổ tổ hợp. Máy tính không thể duyệt toàn bộ cây trò chơi từ đầu đến cuối ván để tìm nước đi tối ưu.

Do đó, bài toán này là mô hình thực nghiệm lý tưởng để áp dụng các thuật toán tìm kiếm đối kháng:

- Minimax giới hạn độ sâu: Giúp AI giả lập tư duy, dự đoán trước các bước đi của đối phương trong một số lượt nhất định.
- Cắt nhánh Alpha-Beta: Loại bỏ các nhánh tìm kiếm vô vọng để giảm số node cần duyệt, tối ưu hóa thời gian phản hồi.

Việc giải quyết bài toán cờ Caro (luật 4 quân không chặn) là cơ hội để vận dụng thực tế kiến thức về biểu diễn trạng thái, tối ưu hệ số phân nhánh và thiết kế hàm đánh giá heuristic trong AI.

1.2. Mục tiêu đề tài

Mục tiêu chính của đề tài là nghiên cứu lý thuyết tìm kiếm đối kháng và áp dụng thực tế để xây dựng một chương trình máy tính chơi cờ Caro hoàn chỉnh. Cụ thể, đề tài hướng tới các mục tiêu sau:

- Cài đặt logic trò chơi: Xây dựng bàn cờ kích thước tối thiểu 9×9 và hiện thực hóa chính xác luật chơi 4 quân liên tiếp chiến thắng, không áp dụng luật chặn hai đầu theo đúng yêu cầu đề bài.
- Xây dựng AI Agent cơ bản (Level 1): Thiết kế tác tử AI sử dụng thuật toán Minimax có giới hạn độ sâu, kết hợp hàm đánh giá heuristic để tự động đưa ra nước đi tối ưu khi đấu với người chơi.
- Tối ưu hóa hiệu năng (Level 2): Nâng cấp thuật toán bằng kỹ thuật cắt nhánh Alpha-Beta nhằm giảm thiểu không gian tìm kiếm, tăng tốc độ xử lý của AI mà không làm thay đổi kết quả nước đi.

- Đo đạc và đánh giá thực nghiệm (Level 3): Tiến hành chạy thử nghiệm trên ít nhất 5 trạng thái bàn cờ tĩnh để thu thập số liệu, từ đó phân tích định lượng về số node đã duyệt và thời gian phản hồi giữa hai thuật toán.

1.3. Các mức độ và yêu cầu chức năng của chương trình

Để đáp ứng các tiêu chí kiểm thử và đánh giá thuật toán, chương trình cần hoàn thiện các yêu cầu chức năng và kỹ thuật sau:

- Chế độ chơi (Game Modes): Hỗ trợ chế độ Người đấu với Máy (Player vs AI). Người chơi và Máy tính luân phiên thực hiện nước đi theo đúng quy tắc.
- Cấu hình thuật toán: Giao diện cho phép người dùng tùy chọn linh hoạt giữa hai thuật toán tìm kiếm: Minimax thuần túy hoặc Minimax cải tiến với cắt nhánh Alpha-Beta.
- Quản lý độ sâu (Search Depth): Cho phép thiết lập giới hạn độ sâu tìm kiếm ($D = 1, 2, 3, \dots$) trước khi bắt đầu ván đấu hoặc trước mỗi lượt đi của Máy nhằm kiểm soát không gian cây trò chơi.
- Đo đạc và hiển thị thông số: Hệ thống phải tự động ghi nhận, đo lường và hiển thị tường minh các chỉ số thực nghiệm ngay sau khi Máy hoàn thành nước đi, bao gồm: tọa độ nước đi được chọn, giá trị hàm đánh giá (Score), độ sâu tìm kiếm, tổng số trạng thái (node) đã duyệt, và thời gian thực thi (tính bằng mili-giây).
- Giao diện và tương tác: Triển khai trên nền tảng đồ họa đơn giản (hoặc môi trường Console trực quan), đảm bảo hiển thị rõ ràng trạng thái bàn cờ, lượt đi hiện tại, và cập nhật tức thời kết quả khi đạt trạng thái kết thúc (Thắng/Thua/Hòa).

Phần 2: Phân tích và thiết kế thuật toán

2.1. Luật chơi và điều kiện kết thúc

Chương trình hiện thực hóa trò chơi cờ Caro dựa trên các quy tắc đóng gói trạng thái và ràng buộc sau:

- Không gian bàn cờ: Trò chơi diễn ra trên một lưới ma trận vuông có kích thước tối thiểu từ 9×9 trở lên. Các ô cờ ban đầu đều ở trạng thái trống và chưa có quân.
- Quy trình thực hiện lượt: Hai thực thể (Người chơi và Máy tính) luân phiên thực hiện nước đi của mình vào các vị trí hợp lệ trên bàn cờ. Một ô đã có quân cờ thì không thể tái sử dụng hoặc đánh đè lên trong suốt ván đấu.
- Ký hiệu tác tử: Theo đặc tả của bài toán, các ô trống trên bàn cờ được biểu diễn bằng ký hiệu dấu chấm (.), Người chơi sử dụng ký hiệu X, và Máy tính sử dụng ký hiệu O.
- Điều kiện dừng và phân định thắng thua: Trò chơi đạt trạng thái kết thúc (Terminal State) khi một trong hai bên tích lũy đủ 4 quân cờ giống nhau nằm liên tiếp theo hàng ngang, hàng dọc, hoặc theo hai đường chéo. Hệ thống không xét luật chặn hai đầu, nghĩa là chỉ cần xuất hiện chuỗi 4 quân hợp lệ thì người chơi đó lập tức được tính là chiến thắng. Trường hợp toàn bộ các ô trên bàn cờ đã được lấp đầy nhưng không có bên nào thỏa mãn điều kiện thắng, ván đấu được ghi nhận kết quả hòa.

2.2. Biểu diễn trạng thái bàn cờ (State Representation)

Để lưu trữ và thao tác trên cấu trúc dữ liệu của trò chơi, trạng thái bàn cờ (State) tại mỗi thời điểm được biểu diễn bằng một ma trận hai chiều (mảng hai chiều) có kích thước cố định 15×15 .

Mỗi phần tử trong ma trận ánh xạ trực tiếp đến một tọa độ (r, c) trên lưới đồ họa và lưu trữ ký hiệu dạng chuỗi (string) để phản ánh trạng thái của ô cờ:

- '.': Đại diện cho ô trống chưa có quân.
- 'X': Đại diện cho vị trí đã đánh của Người chơi.
- 'O': Đại diện cho vị trí đã đánh của Máy tính (AI).

Mã nguồn khởi tạo cấu trúc dữ liệu bàn cờ được triển khai như sau:

```
# Khởi tạo trạng thái ban đầu của bàn cờ kích thước 15x15 với toàn bộ ô trống
self.board = [['.' for _ in range(self.size)] for _ in range(self.size)]
```

2.3. Cơ chế sinh nước đi hợp lệ và tối ưu không gian tìm kiếm

- Nguyên lý sinh nước đi: Trình tự duyệt và tối ưu hóa các nước đi khả thi được thực hiện tập trung qua hàm `get_ordered_moves()`. Trường hợp bàn cờ hoàn toàn trống (chưa có quân cờ nào được đánh), hệ thống lập tức trả về tọa độ trung tâm bàn cờ (`self.size // 2, self.size // 2`) để thiết lập thế trận khai cuộc tối ưu.

- Tối ưu giảm hệ số phân nhánh (Branching Factor): Đối với các trạng thái tiếp theo, thay vì phải xét toàn bộ không gian ô trống trên ma trận kích thước 15x15, thuật toán thực hiện lọc lân cận bằng cách chỉ ghi nhận các ô trống '.' có ít nhất một ô xung quanh (trong phạm vi 8 hướng) đã chứa quân cờ. Cơ chế này loại bỏ hoàn toàn các vùng trống vô nghĩa ở xa chiến trường, giảm thiểu đáng kể số lượng node cần duyệt trên cây trò chơi.
- Sắp xếp nước đi (Move Ordering): Các ô trống hợp lệ được tích lũy điểm ưu tiên (neighbor_score) dựa trên mật độ quân cờ xung quanh. Danh sách nước đi sau đó được sắp xếp giảm dần theo điểm số lân cận. Việc ưu tiên duyệt các nước đi có điểm cao trước giúp thuật toán cắt nhánh Alpha-Beta phát huy tối đa hiệu năng loại bỏ sớm các nhánh phụ.

Mã nguồn xử lý sinh và sắp xếp nước đi trích xuất từ chương trình:

```
def get_ordered_moves(self):
    move_scores = []
    has_pieces = False

    for r in range(self.size):
        for c in range(self.size):
            if self.board[r][c] != '.':
                has_pieces = True
                break
        if has_pieces:
            break

    if not has_pieces:
        return [(self.size // 2, self.size // 2)]

    for r in range(self.size):
        for c in range(self.size):
            if self.board[r][c] == '.':
                neighbor_score = 0
                for dr in [-1, 0, 1]:
                    for dc in [-1, 0, 1]:
                        if dr == 0 and dc == 0:
                            continue
                        nr, nc = r + dr, c + dc
                        if 0 <= nr < self.size and 0 <= nc < self.size:
                            if self.board[nr][nc] != '.':
                                neighbor_score += 1
                if neighbor_score > 0:
                    move_scores.append(((r, c), neighbor_score))

    move_scores.sort(key=lambda x: x[1], reverse=True)
    return [move for move, score in move_scores]
```

2.4. Hàm kiểm tra trạng thái kết thúc (Terminal Test)

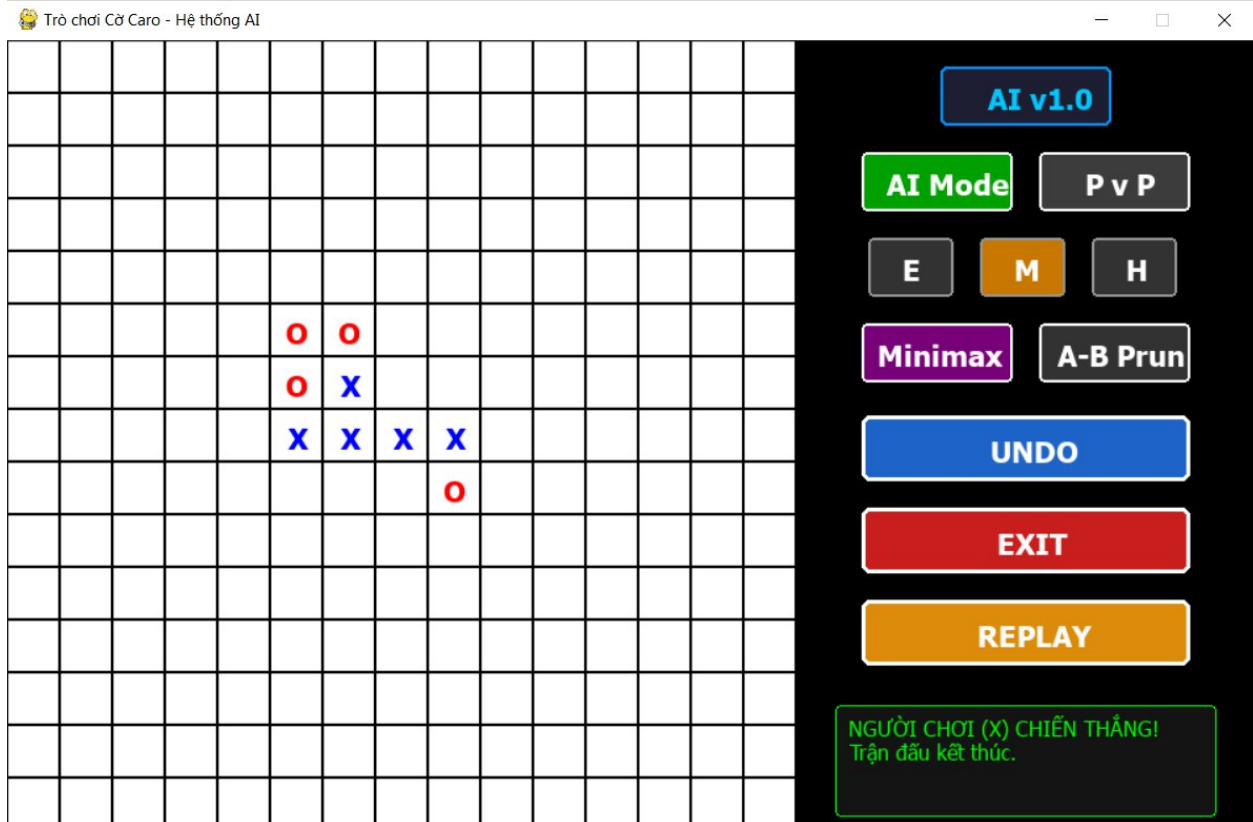
Mục đích của hàm kiểm tra trạng thái kết thúc là xác định xem ván đấu đã phân định thắng thua hay đạt trạng thái hòa hay chưa ngay sau mỗi nước đi, từ đó giúp thuật toán Minimax dừng tiến trình duyệt cây tại các node lá.

Trong chương trình, logic này được cấu trúc tập trung trong hàm `check_win(symbol)`. Hệ thống tiến hành quét toàn bộ ma trận 15x15 để phát hiện chuỗi đạt mục tiêu. Theo đặc tả bài toán, điều kiện dừng được phân tách thành hai trường hợp:

- Điều kiện chiến thắng: Xuất hiện một chuỗi gồm đúng 4 quân liên tiếp có cùng ký hiệu (symbol nhận giá trị 'X' hoặc 'O') theo một trong bốn hướng: hàng ngang, hàng dọc, đường chéo xuôi, hoặc đường chéo ngược. Thuật toán tuân thủ nghiêm ngặt yêu cầu không xét luật chặn hai đầu, nghĩa là chỉ cần kiểm tra đủ 4 ô liên tiếp đồng nhất là trả về trạng thái thắng cuộc.
- Điều kiện hòa cờ: Được kiểm tra thông qua hàm hỗ trợ `is_full()`. Nếu tất cả 15x15 ô trên ma trận đều đã được lấp đầy (không còn chứa ký hiệu '.') và hàm `check_win` không ghi nhận bất kỳ thực thể nào chiến thắng, ván đấu lập tức kết thúc với kết quả hòa.

Mã nguồn xử lý kiểm tra trạng thái chiến thắng được định dạng chuẩn hóa như sau:

```
def check_win(self, symbol):
# Quét theo hàng ngang
    for r in range(self.size):
        for c in range(self.size - 3):
            if all(self.board[r][c+i] == symbol for i in range(4)): return True
# Quét theo hàng dọc
    for r in range(self.size - 3):
        for c in range(self.size):
            if all(self.board[r+i][c] == symbol for i in range(4)): return True
# Quét theo đường chéo xuôi
    for r in range(self.size - 3):
        for c in range(self.size - 3):
            if all(self.board[r+i][c+i] == symbol for i in range(4)): return True
# Quét theo đường chéo ngược
    for r in range(3, self.size):
        for c in range(self.size - 3):
            if all(self.board[r-i][c+i] == symbol for i in range(4)): return True
    return False
```

Hình 1. Giao diện kết thúc 1 ván cờ

2.5. Hàm đánh giá trạng thái (Heuristic Evaluation Function)

Đối với trò chơi cờ Caro có không gian trạng thái bùng nổ tổ hợp, thuật toán duyệt cây bắt buộc phải giới hạn độ sâu tìm kiếm để đảm bảo thời gian phản hồi thời gian thực. Tại các node lá của độ sâu giới hạn, ván đấu thường chưa kết thúc; do đó, hệ thống cần một hàm đánh giá heuristic nhằm ước lượng định lượng thế trận và gán một giá trị điểm số (Score) cụ thể cho trạng thái đó từ góc nhìn của AI (quân cờ 'O').

Trong chương trình, hàm đánh giá toàn cục `evaluate_board()` hoạt động theo nguyên lý chia cắt bàn cờ 15x15 thành các cấu trúc tuyến tính có độ dài cố định bằng 4 ô (line). Hàm tiến hành quét dọc, quét ngang và quét hai đường chéo để trích xuất toàn bộ các chuỗi 4 ô liên tiếp, sau đó chuyển giao các chuỗi này cho hàm hỗ trợ `evaluate_line()` để tính toán tích lũy điểm số dựa trên các cấu hình chiến lược sau:

- Trạng thái kết thúc trận đấu tuyệt đối: Nếu xuất hiện chuỗi 4 quân 'O' liên tiếp, AI đạt trạng thái thắng cuộc và nhận +100.000 điểm. Ngược lại, nếu xuất hiện chuỗi 4 quân 'X' của đối phương, hệ thống nhận -100.000 điểm.
- Trạng thái chiến lược chủ động (Tấn công): Khi phát hiện chuỗi có lợi cho AI bao gồm 3 quân 'O' và 1 ô trống '.', hệ thống tích lũy +500 điểm. Chuỗi gồm 2 quân 'O' và 2 ô trống '.' được cộng +50 điểm.

- Trạng thái chiến lược bị động (Phòng thủ khẩn cấp): Khi phát hiện đối phương (Người chơi) đang sở hữu chuỗi tiềm năng gồm 3 quân 'X' và 1 ô trống '.', hệ thống sẽ trừ đi một lượng điểm rất lớn là -900 điểm. Chuỗi gồm 2 quân 'X' và 2 ô trống '.' của Người chơi sẽ bị trừ -80 điểm.

Ý nghĩa toán học và chiến lược: Trọng số tuyệt đối của việc ngăn chặn đối phương tạo chuỗi thắng (|-900|) được thiết lập lớn hơn hẳn so với trọng số điểm tự tạo chuỗi tấn công (|+500|). Sự chênh lệch này định hình nên một phong cách chơi thực tế cho AI: Luôn ưu tiên đặt tính an toàn và phòng thủ lên hàng đầu, lập tức di chuyển đến các vị trí trọng yếu để chặn đứng nước đi quyết định của Người chơi trước khi nghĩ đến việc thiết lập thế trận tấn công của riêng mình.

Mã nguồn xử lý lượng hóa điểm số cho từng đường tuyến tính được định dạng chuẩn hóa như sau:

```
def evaluate_line(self, line, ai_s, pl_s):
    score = 0
    ai_c = line.count(ai_s)
    pl_c = line.count(pl_s)
    em_c = line.count('.')
    # Trạng thái kết thúc trận đấu tuyệt đối
    if ai_c == 4: return 100000
    if pl_c == 4: return -100000
    # Trạng thái chiến lược tấn công và phòng thủ
    if ai_c == 3 and em_c == 1: score += 500
    elif pl_c == 3 and em_c == 1: score -= 900
    if ai_c == 2 and em_c == 2: score += 50
    elif pl_c == 2 and em_c == 2: score -= 80
    return score
```

2.6. Cài đặt thuật toán Minimax

Thuật toán Minimax được triển khai trong chương trình thông qua hàm minimax() nhằm xác định nước đi tối ưu cho Máy tính (tác tử MAX) trong trò chơi đối kháng có tổng bằng không. Về mặt lý thuyết, thuật toán duyệt qua cây trò chơi bằng cách giả lập chuỗi nước đi luân phiên giữa hai thực thể cho đến khi chạm giới hạn độ sâu cấu hình (depth), từ đó thu thập giá trị lượng hóa thế trận từ hàm đánh giá nhằm đưa ra quyết định di chuyển.

Cơ chế vận hành của thuật toán bao gồm hai tiến trình xử lý đối lập:

- Trạng thái MAX (Lượt của AI - quân 'O'): Tìm cách tối đa hóa điểm số nhận được từ hàm đánh giá heuristic để đưa AI đến các thế cờ có lợi nhất.
- Trạng thái MIN (Lượt của Người chơi - quân 'X'): Giả định đối phương sẽ luôn thực hiện nước đi thông minh nhất để cực tiểu hóa điểm số của AI, dồn AI vào thế bất lợi.

Trong mã nguồn thực tế, hàm minimax nhận vào ba tham số: độ sâu tìm kiếm (depth), biến logic is_maximizing để phân định lượt duyệt của MAX/MIN, và một danh sách chứa một phần tử nodes_counter đóng vai trò làm bộ đếm tích lũy số trạng thái đã duyệt phục vụ cho công tác thực nghiệm. Tiến trình đệ quy sẽ dừng lại và trả về kết quả định lượng của hàm evaluate_board() khi độ sâu chạm ngưỡng 0 (depth == 0) hoặc ván đấu kết thúc sớm (check_win hoặc is_full).

Mã nguồn cài đặt cấu trúc đệ quy của thuật toán Minimax được trích xuất chính xác từ chương trình:

```
def evaluate_board(self):
    score = 0
    ai_s, pl_s = 'O', 'X'
    for def minimax(self, depth, r in range(self.size):
        for c in range(self.size - 3):
            score += self.evaluate_line([self.board[r][c+i] for i in range(4)], ai_s, pl_s)
    for r in range(self.size - 3):
        for c in range(self.size):
            score += self.evaluate_line([self.board[r+i][c] for i in range(4)], ai_s, pl_s)
    for r in range(self.size - 3):
        for c in range(self.size - 3):
            score += self.evaluate_line([self.board[r+i][c+i] for i in range(4)], ai_s, pl_s)
    for r in range(3, self.size):
        for c in range(self.size - 3):
            score += self.evaluate_line([self.board[r-i][c+i] for i in range(4)], ai_s, pl_s)
    return score

is_max):
    self.states_cnt += 1
    if self.check_win('O'): return 100000 + depth
    if self.check_win('X'): return -100000 - depth
    if self.is_full() or depth == 0: return self.evaluate_board()

valid_moves = self.get_ordered_moves()
if is_max:
    best = -float('inf')
    for r, c in valid_moves:
        self.board[r][c] = 'O'
        best = max(best, self.minimax(depth - 1, False))
        self.board[r][c] = '.'
    return best
else:
    best = float('inf')
    for r, c in valid_moves:
        self.board[r][c] = 'X'
        best = min(best, self.minimax(depth - 1, True))
```

```
self.board[r][c] = '.'  
return best
```

2.7. Cài đặt thuật toán cải tiến Alpha-Beta Pruning

Thuật toán cắt nhánh Alpha-Beta (Alpha-Beta Pruning) được cài đặt qua hàm `alphabeta()` nhằm mục đích tối ưu hóa hiệu năng và tốc độ xử lý cho cây tìm kiếm Minimax. Do không gian trạng thái của bàn cờ 15x15 rất lớn, việc duyệt toàn bộ các nhánh của cây trò chơi ở các độ sâu cao là không khả thi về mặt thời gian. Kỹ thuật này giúp loại bỏ sớm các nhánh tìm kiếm vô vọng - những nhánh chắc chắn không mang lại kết quả tốt hơn các phương án đã được xem xét trước đó - mà vẫn đảm bảo trả về nước đi tối ưu tương đương với Minimax thuần túy.

Cơ chế vận hành của thuật toán dựa trên việc duy trì hai tham số giới hạn trong suốt quá trình đệ quy:

- α (Alpha): Giá trị điểm số tối thiểu mà tác tử MAX (AI) chắc chắn đã bảo toàn được từ các nhánh độc lập trước đó. Ban đầu α được khởi tạo bằng $-\infty$.
- β (Beta): Giá trị điểm số tối đa mà tác tử MIN (Người chơi) có thể ép AI phải nhận, dựa trên các phương án đã duyệt. Ban đầu β được khởi tạo bằng $+\infty$.

Điều kiện cắt nhánh xuất hiện khi hệ thống phát hiện mối quan hệ $\beta \leq \alpha$. Tại một node MAX, nếu điểm lượng hóa của nhánh con hiện tại vượt quá hoặc bằng giá trị β hiện tại, điều đó có nghĩa là người chơi (MIN) ở node cha sẽ không bao giờ cho phép trận đấu đi vào nhánh này. Do đó, thuật toán lập tức dừng việc duyệt các nước đi còn lại của node đó (break). Quá trình tương tự diễn ra đối với các node MIN khi điểm số giảm xuống dưới hoặc bằng giá trị α .

Mã nguồn cài đặt cấu trúc đệ quy của thuật toán Alpha-Beta được trích xuất chính xác từ chương trình:

```
def alphabeta(self, depth, alpha, beta, is_max):  
    self.states_cnt += 1  
    if self.check_win('O'): return 100000 + depth  
    if self.check_win('X'): return -100000 - depth  
    if self.is_full() or depth == 0: return self.evaluate_board()  
  
    valid_moves = self.get_ordered_moves()  
    if is_max:  
        best = -float('inf')  
        for r, c in valid_moves:  
            self.board[r][c] = 'O'  
            score = self.alphabeta(depth - 1, alpha, beta, False)  
            self.board[r][c] = '.'  
            best = max(best, score)  
            alpha = max(alpha, score)
```

```

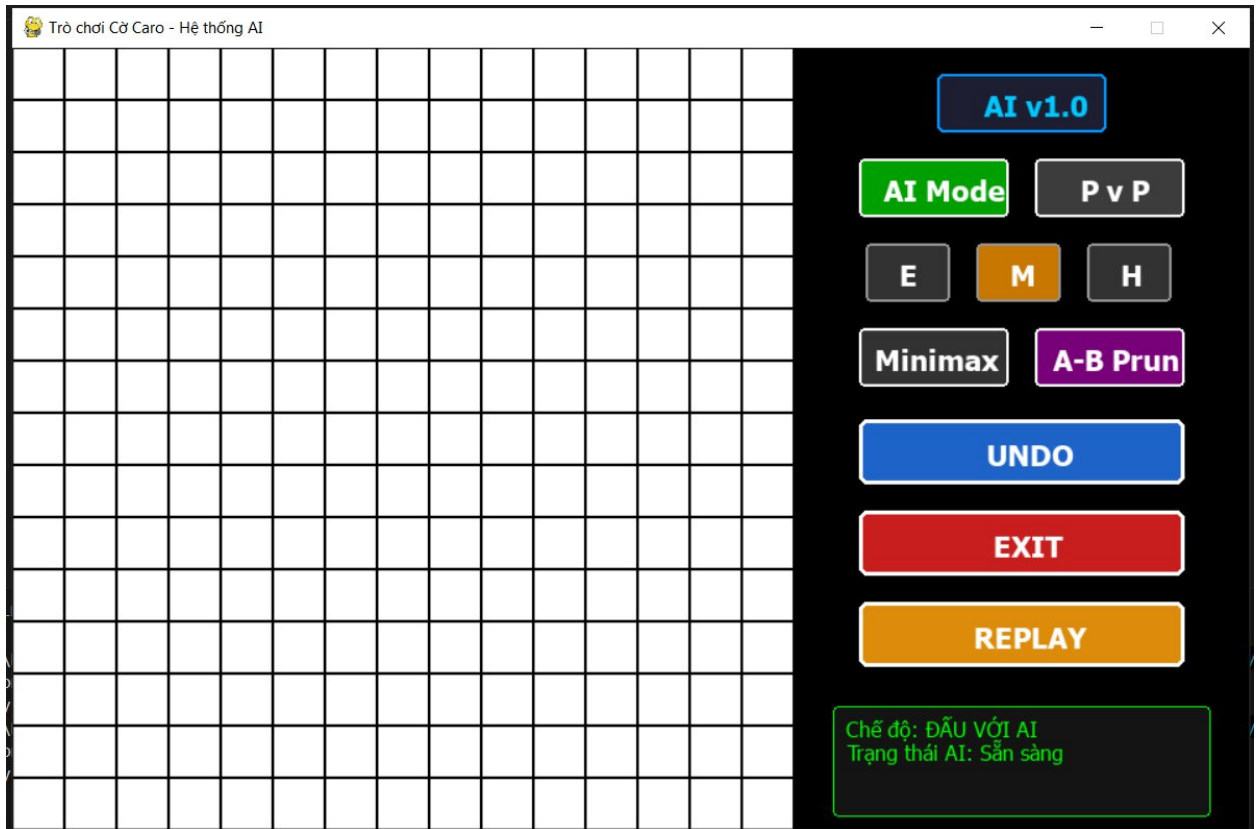
        if beta <= alpha: break
    return best
else:
    best = float('inf')
    for r, c in valid_moves:
        self.board[r][c] = 'X'
        score = self.alphabeta(depth - 1, alpha, beta, True)
        self.board[r][c] = '.'
        best = min(best, score)
        beta = min(beta, score)
        if beta <= alpha: break
    return best

```

2.8. Thiết kế giao diện chương trình

Giao diện đồ họa của chương trình được phát triển trên nền tảng thư viện Pygame, tổ chức phân bố không gian hiển thị trực quan thông qua hàm `draw_board(game)`. Cấu trúc giao diện được phân tách thành hai phân vùng chức năng cốt lõi:

- Không gian bàn cờ thi đấu: Biểu diễn ma trận lưới kích thước 15x15 ô vuông dựa trên các tham số cấu hình hệ thống (`BOARD_SIZE`, `CELL_SIZE`). Quân cờ của Người chơi (X) được hiển thị bằng màu xanh lam (`COLOR_X`), trong khi quân cờ của Máy tính (O) được định vị bằng màu đỏ (`COLOR_O`).
- Thanh công cụ điều khiển bên phải (Sidebar): Nền tối (`COLOR_SIDEBAR`), chứa hệ thống nút tương tác vật lý được dựng bằng các khối hình chữ nhật vẽ trực tiếp qua phương thức `pygame.draw.rect()`. Trạng thái kích hoạt của các nút (như chế độ chơi, thuật toán sử dụng, độ sâu tìm kiếm) được phản ánh động thông qua việc thay đổi màu sắc hiển thị. Khối nhật ký hệ thống ở góc dưới sử dụng vòng lặp để duyệt và in danh sách thông điệp `game.log_msg`.



Hình 2. Giao diện trò chơi

Phần 3: Thực Nghiệm và phân tích đánh giá

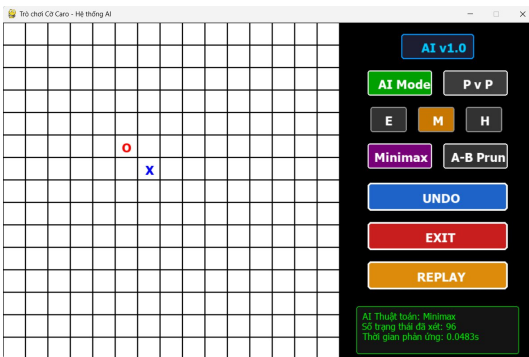
3.1. Thiết kế các kịch bản điểm thử

Để đánh giá hiệu quả của hai thuật toán Minimax và Alpha-Beta pruning, nhóm đã chuẩn bị ít nhất 5 trạng thái bàn cờ tĩnh với đặc trưng khác nhau. Các trạng thái này được lựa chọn nhằm kiểm tra khả năng tấn công, phòng thủ và xử lý tình huống phức tạp của AI.

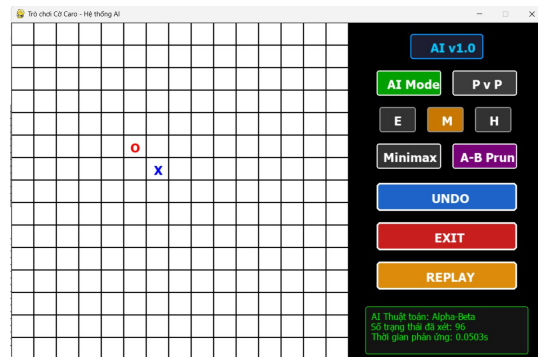
a) Trạng thái 1: Đầu ván

Trong trạng thái này, người chơi đánh quân **X** đầu tiên, sau đó AI đánh quân **O** ở một vị trí gần đó. Mục tiêu của kịch bản là quan sát cách AI phản ứng ngay từ đầu: liệu AI sẽ chọn nước đi tiếp theo để phát triển thế tấn công hay ưu tiên phòng thủ.

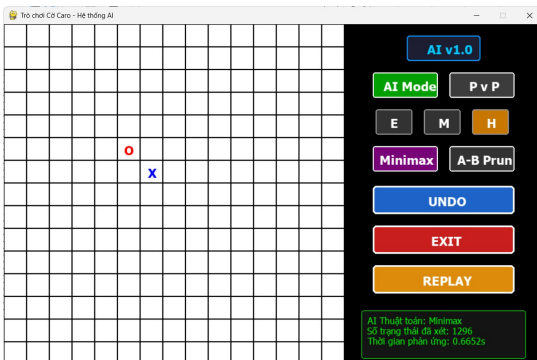
- Ý nghĩa kiểm thử:
 - Kiểm tra khả năng AI xây dựng thế cờ từ tình huống đơn giản.
 - Đánh giá xem Minimax và Alpha-Beta có chọn cùng nước đi hay không.
 - So sánh số trạng thái xét và thời gian chạy khi bàn cờ còn rất ít quân.



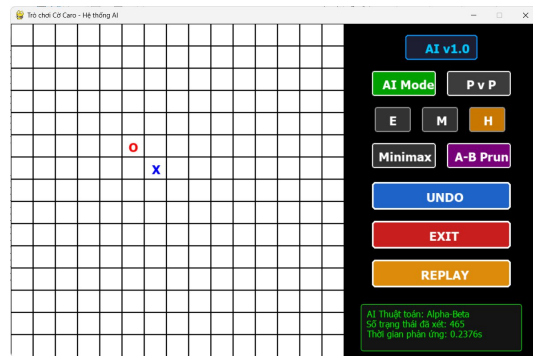
Hình 3.1.a. Trạng thái đầu game hàm Minimax với độ sâu 2



Hình 3.1.b. Trạng thái đầu game với hàm Alpha-Beta, độ sâu 2



Hình 3.1.c. Trạng thái đầu game hàm Minimax với độ sâu 3

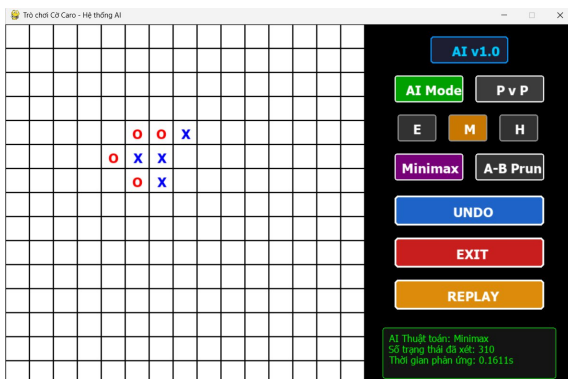


Hình 3.1.d. Trạng thái đầu game với hàm Alpha-Beta, độ sâu 3

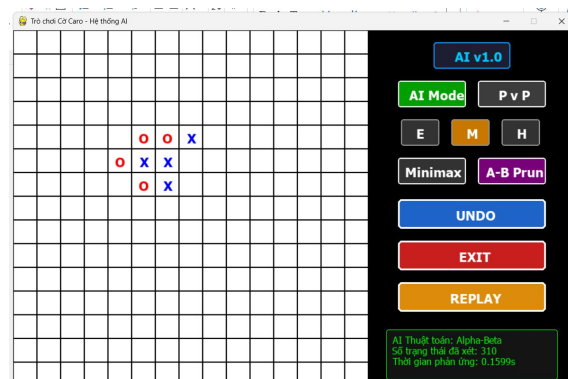
b) Trạng thái 2: Giữa ván

Trong trạng thái này, bàn cờ đã có nhiều quân X và O được đặt, thế trận trở nên giằng co và phức tạp. Mục tiêu của kịch bản là kiểm tra khả năng AI phân tích nhiều nhánh nước đi, đánh giá tình huống tấn công – phòng thủ đồng thời, và lựa chọn nước đi tối ưu trong bối cảnh có nhiều khả năng xảy ra.

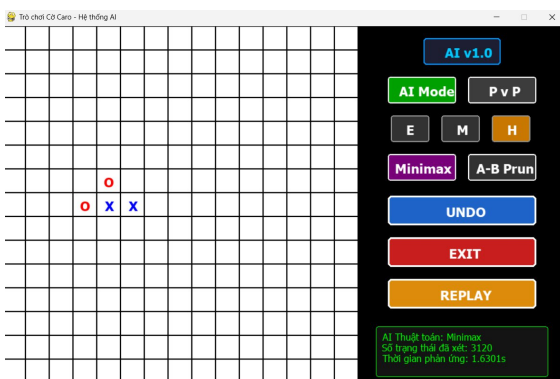
- Ý nghĩa kiểm thử:
 - Kiểm tra khả năng AI xử lý tình huống phức tạp khi bàn cờ không còn đơn giản.
 - Đánh giá xem Minimax và Alpha-Beta có chọn cùng nước đi hay không khi có nhiều lựa chọn khả thi.
 - So sánh số trạng thái xét và thời gian chạy trong tình huống “giữa ván” – nơi số lượng nhánh tăng mạnh.



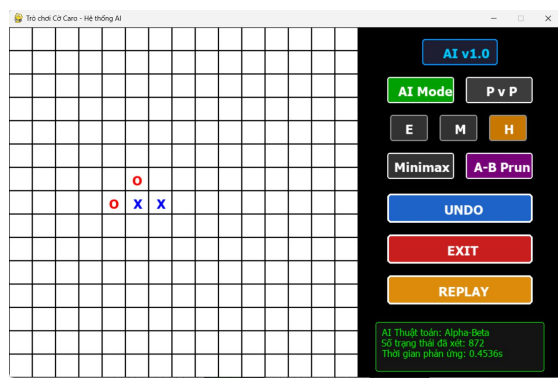
Hình 3.2.a. Trạng thái giữa game với hàm Minimax, độ sâu 2



Hình 3.2.b. Trạng thái giữa game với hàm Alpha-Beta, độ sâu 2



Hình 3.2.c. Trạng thái giữa game với hàm Minimax, độ sâu 3

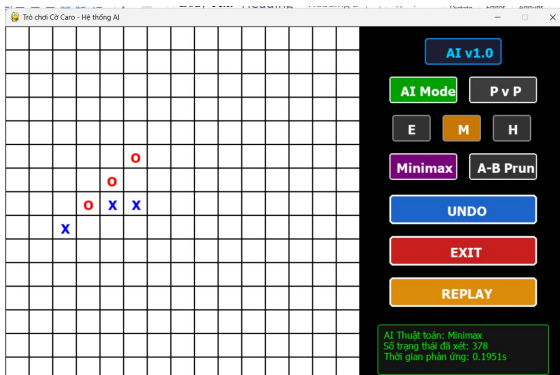


Hình 3.2.d. Trạng thái giữa game với hàm Alpha-Beta, độ sâu 3

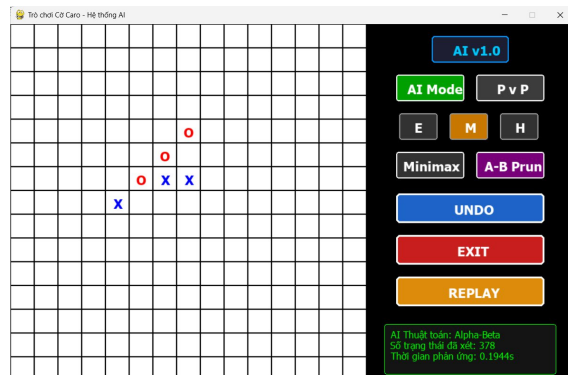
c) Trạng thái 3: Tấn công thẳng ngay

Trong trạng thái này, AI đã có sẵn một chuỗi OOO liên tiếp theo hàng ngang/dọc/chéo, và chỉ còn một ô trống để hoàn thành chuỗi 4 quân thắng. Đây là tình huống điển hình để kiểm tra khả năng tấn công dứt điểm của AI.

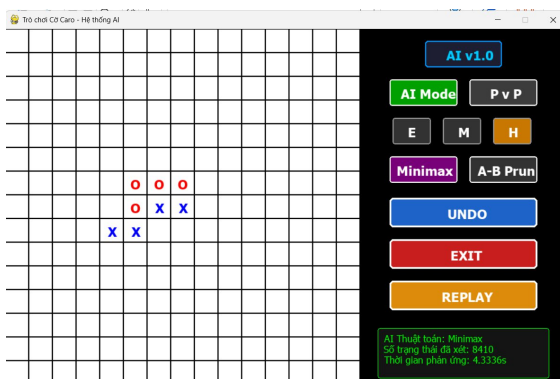
- Ý nghĩa kiểm thử:
 - Đánh giá xem AI có nhận diện được cơ hội thắng ngay lập tức.
 - Kiểm tra sự khác biệt về hiệu suất giữa Minimax và Alpha-Beta pruning khi cùng chọn nước đi quyết định.
 - Nếu AI không chọn nước đi thắng, chứng tỏ hàm đánh giá hoặc thuật toán cài đặt chưa đúng.



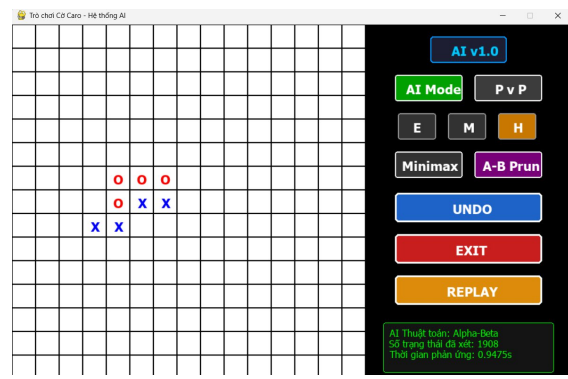
Hình 3.3.a. Trạng thái AI tấn công thẳng với hàm Minimax, độ sâu 2



Hình 3.3.b. Trạng thái AI tấn công thẳng với hàm Alpha-Beta, độ sâu 2



Hình 3.3.c. Trạng thái AI tấn công thẳng với hàm Minimax, độ sâu 3

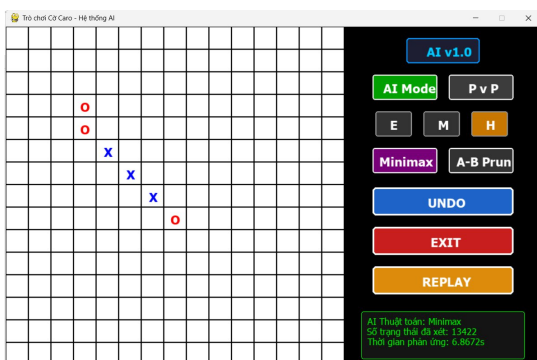


Hình 3.3.d. Trạng thái AI tấn công thẳng với hàm Alpha-Beta, độ sâu 3

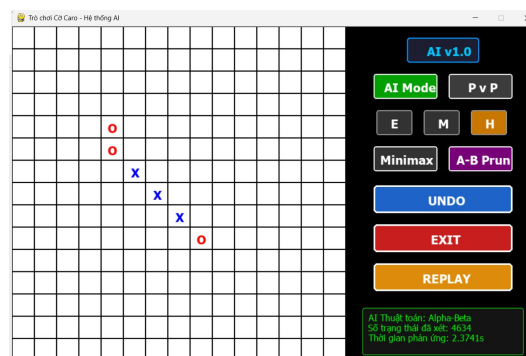
d) Trạng thái 4: Phòng thủ khẩn cấp

Trong trạng thái này, người chơi đã có chuỗi XXX liên tiếp, chỉ còn một ô trống để hoàn thành chuỗi 4 quân thắng. Đây là tình huống bắt buộc AI phải phòng thủ, nếu không sẽ thua ngay lập tức.

- Ý nghĩa kiểm thử:
 - Kiểm tra khả năng AI nhận diện tình huống nguy hiểm và chọn nước đi phòng thủ bắt buộc.
 - Đánh giá sự khác biệt về hiệu suất giữa Minimax và Alpha-Beta khi cùng chọn nước chặn.
 - Nếu AI không chọn nước chặn, chứng tỏ thuật toán hoặc hàm đánh giá chưa đúng.



Hình 3.4.a. Trạng thái AI phòng thủ khẩn cấp với hàm Minimax, độ sâu 3



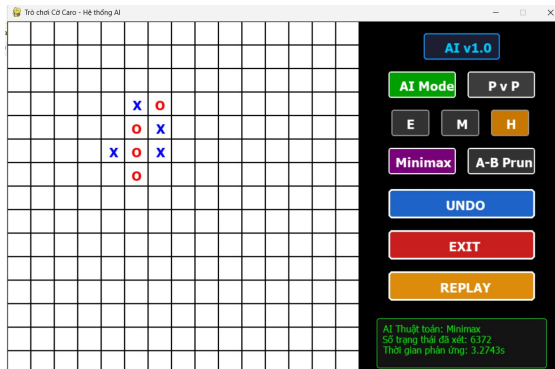
Hình 3.4.b. Trạng thái AI phòng thủ khẩn cấp với hàm Alpha-Beta, độ sâu 3

e) Trạng thái 5: Tranh chấp ngôi nổi

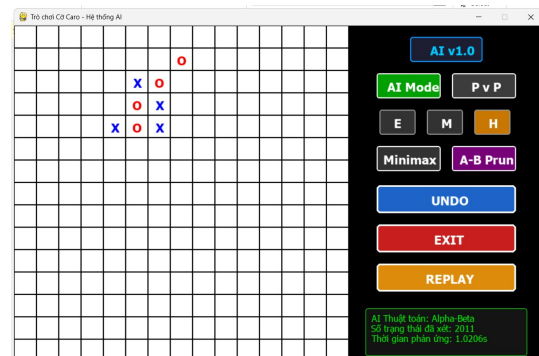
Trong trạng thái này, cả hai bên (người chơi và AI) đều đang xây dựng các chuỗi 2–3 quân liên tiếp, tạo ra nhiều hướng tấn công tiềm năng. AI cần lựa chọn nước đi tối ưu để vừa ngăn chặn đối thủ, vừa phát triển thế cờ của mình. Đây là tình huống phức tạp, đòi hỏi thuật toán phải cân bằng giữa tấn công và phòng thủ.

- Ý nghĩa kiểm thử:
 - Kiểm tra khả năng AI phân tích nhiều hướng đi cùng lúc.
 - Đánh giá xem Minimax và Alpha-Beta có chọn cùng một nước đi hay khác nhau trong tình huống phức tạp.

- So sánh hiệu suất: số trạng thái xét và thời gian chạy khi bàn cờ có nhiều nhánh khả thi.
- Đây là trạng thái giúp thấy rõ ưu thế của Alpha-Beta khi số lượng nhánh tăng mạnh.



Hình 3.5.a. Trạng thái AI tranh chấp ngôi nỏ với hàm Minimax, độ sâu 3



Hình 3.5.b. Trạng thái AI tranh chấp ngôi nỏ với hàm Alpha-Beta, độ sâu 3

3.2. Bảng kết quả thực nghiệm so sánh

Để đánh giá hiệu quả của hai thuật toán Minimax và Alpha-Beta pruning, nhóm đã tiến hành kiểm thử trên 5 trạng thái điển hình của ván cờ Caro với độ sâu 2 và 3. Trong mỗi trạng thái, chương trình tự động ghi nhận:

- Nước đi mà AI lựa chọn
- Số trạng thái đã xét (biến `states_cnt` trong code)
- Thời gian phản ứng (đo bằng `time.time()` trước và sau khi gọi hàm tìm nước đi)

Kết quả số trạng thái đã xét với độ sâu 3 được tổng hợp trong bảng sau:

Trạng thái	Thuật toán	Nước đi chọn	Số trạng thái xét	Thời gian chạy
1. Đầu ván	Minimax	(6,6)	1296	0.6652 s
	Alpha-Beta	(6,6)	465	0.2376 s
2. Giữa ván	Minimax	(8,4)	3120	1.63 s
	Alpha-Beta	(8,4)	872	0.4536 s
3. Tấn công	Minimax	(7,6)	8410	4.3336 s
	Alpha-Beta	(7,6)	1908	0.9475 s
4. Phòng thủ	Minimax	(9,8)	13422	6.8672 s
	Alpha-Beta	(9,8)	4634	2.3741 s
5. Tranh chấp	Minimax	(4,7)	6372	3.2743 s
	Alpha-Beta	(2,8)	2011	1.0206 s

3.3. Nhận xét và phân tích kết quả thực nghiệm

Dựa trên những ảnh trong mục 3.1 và bảng số liệu ở mục 3.2, ta có thể rút ra các phân tích sau:

- Sự đồng nhất về kết quả: Trong cả 5 trạng thái, Minimax và Alpha-Beta đều chọn cùng một nước đi (ví dụ: trạng thái 1 cả hai đều chọn ô (6,6); trạng thái 3 cùng chọn (7,6)). Điều này đúng với lý thuyết: Alpha-Beta chỉ là tối ưu hóa của Minimax nên kết quả cuối cùng phải giống nhau.
- Hiệu quả cắt nhánh: Alpha-Beta giảm đáng kể số trạng thái xét:
 - Trạng thái 1: từ 1296 xuống 465 (giảm ~64%).
 - Trạng thái 3: từ 8410 xuống 1908 (giảm ~77%).
 - Trạng thái 4: từ 13422 xuống 4634 (giảm ~65%). → Trung bình, Alpha-Beta giúp giảm khoảng 60–75% số trạng thái duyệt so với Minimax.
- Sự thay đổi theo độ sâu: Khi tăng độ sâu tìm kiếm lên 3, số trạng thái và thời gian chạy tăng rất nhanh. Ví dụ:
 - Trạng thái 2: Minimax mất 1.63s, Alpha-Beta chỉ 0.45s.
 - Trạng thái 4: Minimax mất 6.86s, Alpha-Beta chỉ 2.37s. → Alpha-Beta giúp thời gian chạy giảm còn khoảng 1/3 đến 1/4 so với Minimax.
- Chất lượng nước đi: Độ sâu lớn hơn giúp AI chọn nước đi hợp lý hơn trong thế trận phức tạp. Ở độ sâu 3, AI có thể nhìn xa hơn, chọn đúng nước tấn công thắng ngay (OOO → OOOO) hoặc phòng thủ khẩn cấp (XXX → chặn). Nếu chỉ dừng ở độ sâu 1–2, AI dễ bỏ lỡ nước đi chiến lược.
- Đánh giá hàm heuristic:
 - Ưu điểm: Nhận diện tốt chuỗi liên tiếp (2, 3 quân), ưu tiên tấn công/phòng thủ rõ ràng.
 - Hạn chế: Trong thế cờ rời rạc hoặc nhiều hướng mở, heuristic còn đơn giản, đôi khi đánh giá chưa chính xác.
- Trường hợp AI chơi tốt: AI chơi tốt khi có chuỗi liên tiếp rõ ràng (ví dụ tấn công thắng ngay hoặc phòng thủ khẩn cấp).
- Trường hợp AI chưa hợp lý: Trong thế trận tranh chấp nhiều hướng, AI đôi khi chọn nước chưa tối ưu vì heuristic chưa đủ tinh vi.
- Hướng cải tiến:
 - Thêm move ordering để Alpha-Beta cắt nhánh hiệu quả hơn.
 - Nâng cấp hàm heuristic để đánh giá tốt hơn các thế cờ rời rạc và chiến lược dài hạn.
 - Áp dụng iterative deepening để cân bằng giữa thời gian chạy và chất lượng nước đi.

3.4. Ưu điểm và hạn chế của chương trình

- Ưu điểm:
 - Logic thuật toán chuẩn xác, kết quả Minimax và Alpha-Beta luôn đồng nhất về nước đi.
 - Giao diện ổn định, trực quan, có log hiển thị rõ ràng (số trạng thái xét, thời gian phản ứng).
 - Thuật toán Alpha-Beta cắt nhánh hiệu quả, giảm trung bình 60–75% số trạng thái duyệt, giúp thời gian chạy nhanh hơn nhiều so với Minimax.

- Hàm heuristic tự xây dựng giúp AI chơi tốt trong các tình huống tấn công thắng ngay hoặc phòng thủ khẩn cấp.
- Chương trình có các chức năng hỗ trợ như Undo, Replay, chuyển đổi chế độ AI/PvP, độ sâu tìm kiếm, giúp dễ dàng kiểm thử.
- Hạn chế:
 - Ở độ sâu thấp (1–2), Alpha-Beta chưa thể hiện rõ sự khác biệt so với Minimax, do cây tìm kiếm nhỏ.
 - Hàm heuristic còn đơn giản, chưa đánh giá tốt các thế cờ rời rạc hoặc chiến lược dài hạn, dẫn đến một số nước đi chưa tối ưu trong thế trận tranh chấp.
 - Thời gian chạy tăng nhanh khi độ sâu lớn, chưa có cơ chế move ordering để tối ưu thêm việc cắt nhánh.
 - Chưa tích hợp các kỹ thuật nâng cao như iterative deepening hay transposition table để cải thiện hiệu suất và chất lượng nước đi.

Phần 4: Kết luận và hướng phát triển

4.1. Kết luận

Qua thực nghiệm trên 5 trạng thái điển hình của ván cờ Caro, có thể khẳng định:

- Thuật toán Minimax và Alpha-Beta pruning luôn cho cùng một nước đi tối ưu, đảm bảo tính đúng đắn về mặt lý thuyết.
- Alpha-Beta pruning vượt trội về hiệu suất, giúp giảm trung bình 60–75% số trạng thái xét và rút ngắn thời gian chạy xuống còn khoảng 1/3–1/4 so với Minimax.
- Độ sâu tìm kiếm càng lớn thì chất lượng nước đi càng tốt, nhưng chi phí tính toán tăng nhanh.
- Hàm heuristic tự xây dựng đã giúp AI chơi khá tốt trong các tình huống tấn công và phòng thủ rõ ràng, song vẫn còn hạn chế trong thế cờ rời rạc hoặc nhiều hướng mở.

4.1. Hướng phát triển nâng cao

Để nâng cao chất lượng và hiệu quả của chương trình, có thể cải tiến theo các hướng sau:

- Tối ưu cắt nhánh: Áp dụng move ordering để Alpha-Beta cắt nhánh mạnh hơn.
- Nâng cấp heuristic: Xây dựng hàm đánh giá phức tạp hơn, có khả năng nhận diện thế cờ rời rạc, thế chiến lược dài hạn, và ưu tiên các nước đi mở rộng thế trận.
- Iterative deepening: Kết hợp tìm kiếm lặp sâu dần để cân bằng giữa thời gian phản ứng và chất lượng nước đi.
- Transposition table: Lưu trữ các trạng thái đã xét để tránh tính toán lặp lại, tăng tốc độ xử lý.
- Giao diện và trải nghiệm: Bổ sung hiển thị trực quan hơn (ví dụ highlight nước đi AI chọn), hoặc thống kê chi tiết để phục vụ nghiên cứu.

Danh sách tài liệu tham khảo

1. Giáo trình trí tuệ nhân tạo.
2. Link 1: <https://github.com/husus/gomokuAI-py>
3. Link 2: https://github.com/MonHauVD/Caro_AI
4. Tài liệu về Minimax và Alpha-Beta Pruning.